

## GENERAL DESCRIPTION

The MC3419 is a small form factor, integrated digital output 3-axis accelerometer with a feature set optimized for cell phones and IoT product motion sensing. Applications include user interface control, gaming motion input, electronic compass tilt compensation for cell phones, game controllers, remote controls and portable media products.

The MC3419 features a dedicated motion block which implements embedded algorithms to support “any motion” and shake detection, tilt/flip and tilt 35 position detection.

The EV3419A is a prebuilt circuit board with MC3419 LGA-12 3-axes sensor. The MC3419 has internal sample rate from 0.5 to 1000 samples / second and measures acceleration with a wide usage range, from +/-2g up to +/-16g, and 16-bit high precision ADC output, which is easy to fit on top of the microcontroller, such as an Arduino. The accelerometer communicates via I2C/SPI and gives out motion detection or sample acquisition conditions to trigger an interrupt toward an MCU.

The sensor data is easily readable by connecting DVDD to 3.3V, GND to ground, and SCL/SDA pins to your Arduino I2C clock and data pin respectively. Download the MC3419 library from GitHub onto the board, run the example sketch, and then

sensor data shortly comes out in raw data count and SI unit accelerometer measurements. An easy-to-use demonstration on EV3419A using the Arduino platform is depicted in this document.

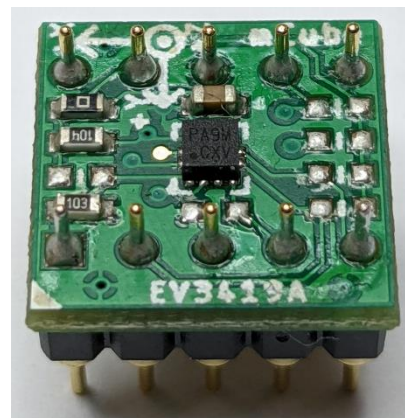
## MC3419 FEATURES

### Range, Sampling & Power

- $\pm 2, 4, 8, 12$  or 16g range
- 16-bit single sample resolution
- 16-bit resolution with FIFO
- 0.5 to 1000 Hz Output Data Rate
- 4  $\mu$ A typical Standby current
- Low typical active current

### Simple System Integration

- I2C interface, up to 1 MHz
- SPI Interface, up to 10 MHz
- 2x2x0.92 mm 12-pin LGA package
- Single-chip 3D silicon MEMS
- RoHS compliant

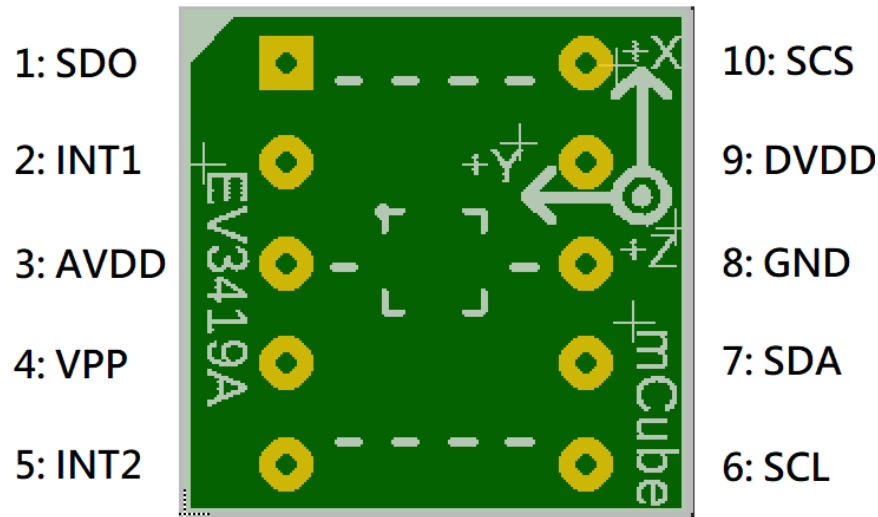


## TABLE OF CONTENTS

|     |                                                            |    |
|-----|------------------------------------------------------------|----|
| 1   | General Operation .....                                    | 3  |
| 1.1 | Pinouts .....                                              | 3  |
| 1.2 | Power Pins .....                                           | 3  |
| 1.3 | I2C Pins .....                                             | 4  |
| 1.4 | SPI Pins .....                                             | 5  |
| 1.5 | Interrupt Pins .....                                       | 5  |
| 2   | Assembly and Test .....                                    | 6  |
| 2.1 | I2C Interface .....                                        | 6  |
| 2.2 | SPI Interface .....                                        | 7  |
| 3   | Demo .....                                                 | 8  |
| 3.1 | Download the Driver from GitHub .....                      | 8  |
| 3.2 | Load the Demo .....                                        | 9  |
| 4   | Library Reference .....                                    | 12 |
| 4.1 | Create MCUBE_MC34X9 Object .....                           | 12 |
| 4.2 | Initialize and Configure Sensor .....                      | 12 |
| 4.3 | Set Range .....                                            | 12 |
| 4.4 | Read Range .....                                           | 12 |
| 4.5 | Set Sampling Rate .....                                    | 12 |
| 4.6 | Read Sampling Rate .....                                   | 13 |
| 4.7 | Config Motion Feature .....                                | 13 |
| 4.8 | Config Interrupt Mode .....                                | 13 |
| 4.9 | Read Raw Count Data .....                                  | 14 |
| 5   | Downloads .....                                            | 15 |
| 5.1 | MC3419 Accelerometer Datasheet and Quick Start Guide ..... | 15 |
| 5.2 | MC34X9 Driver at GitHub .....                              | 15 |
| 5.3 | All Other mCube Documentation .....                        | 15 |
| 6   | Schematics .....                                           | 16 |
| 7   | Bill of Materials .....                                    | 17 |
| 8   | Fabrication Print .....                                    | 18 |
| 9   | Revision History .....                                     | 19 |
| 10  | Legal .....                                                | 20 |

# 1 GENERAL OPERATION

## 1.1 PINOUTS



## 1.2 POWER PINS

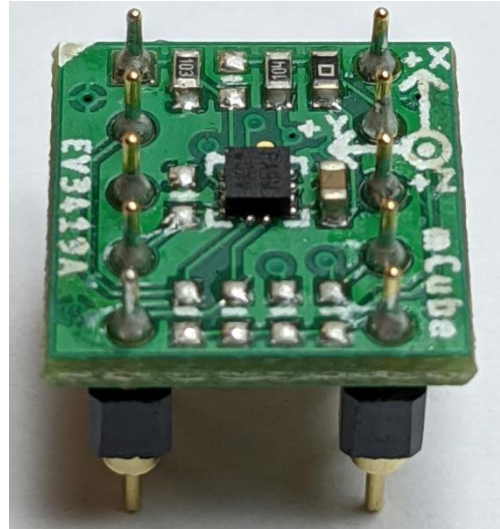
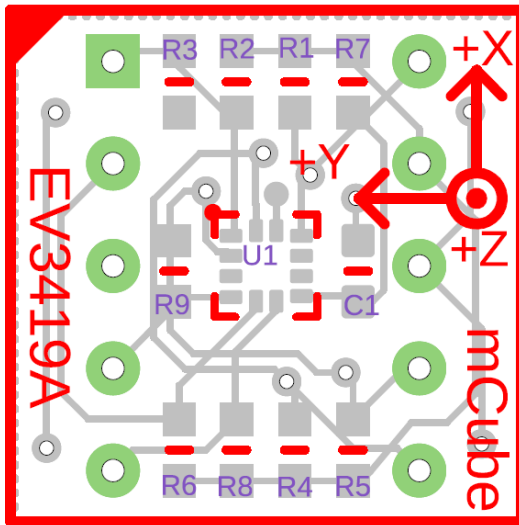
- **DVDD** – 3.3V Power Supply Input
- **GND** – Ground Pin for Power and Logic
- **R7**: The current draw from the sensor can be measured by putting an ammeter in place of R7.
- In the following demonstration, an Arduino DUE is used to illustrate on how to test the evaluation board with a microcontroller.
- Please be advised that if an Arduino UNO is used instead, hardware modification on Arduino UNO **MUST** be made for it to output at 3.3V. (WARNING: attempting to power the part at 5V is likely to damage it.)

By default, Arduino UNO operates at 5V, which is higher than the maximum voltage rating for the evaluation board. Please refer to an excellent tutorial on modifying Arduino UNO to output at 3.3V:

<https://learn.adafruit.com/arduino-tips-tricks-and-techniques/3-3v-conversion>

### 1.3 I2C PINS

- Connect the **SCL** (I2C clock pin) to your microcontroller's I2C clock line.
- Connect the **SDA** (I2C data pin) to your microcontroller's I2C data line.



**R4, R5:** If using I2C and I2C pull-up resistors are needed for your application then install ~4.7K $\Omega$  resistors into R4 (SCL clock pin) and R5 (SDA data pin) which are not installed by factory default. In addition, besides soldering resistors on R4/R5, you can add axial lead 4.7K ohm resistors to the SDA and SCL pin respectively. It will work the same either way.

**NOTE:** DO NOT install more than one setup pull-up resistors per I2C bus.

## 1.4 SPI PINS

With an SPI connection, there is always one master device (usually a microcontroller) which controls the peripheral devices. Typically, there are four wires commonly connected to all the devices:

Connect the **SCS** (Slave Chip Select) to the pin on the device that the master can use to enable and disable SPI cycles.

Connect the **SCL** (Serial clock) to the pin where the clock pulses synchronize data transmission generated by the master

Connect **SDO** to the pin where the Slave sends data to the master (Master Input, Slave Output).

Connect **SDA** to the pin where the Master sends data to the peripherals (Master Output, Slave Input).

## 1.5 INTERRUPT PINS

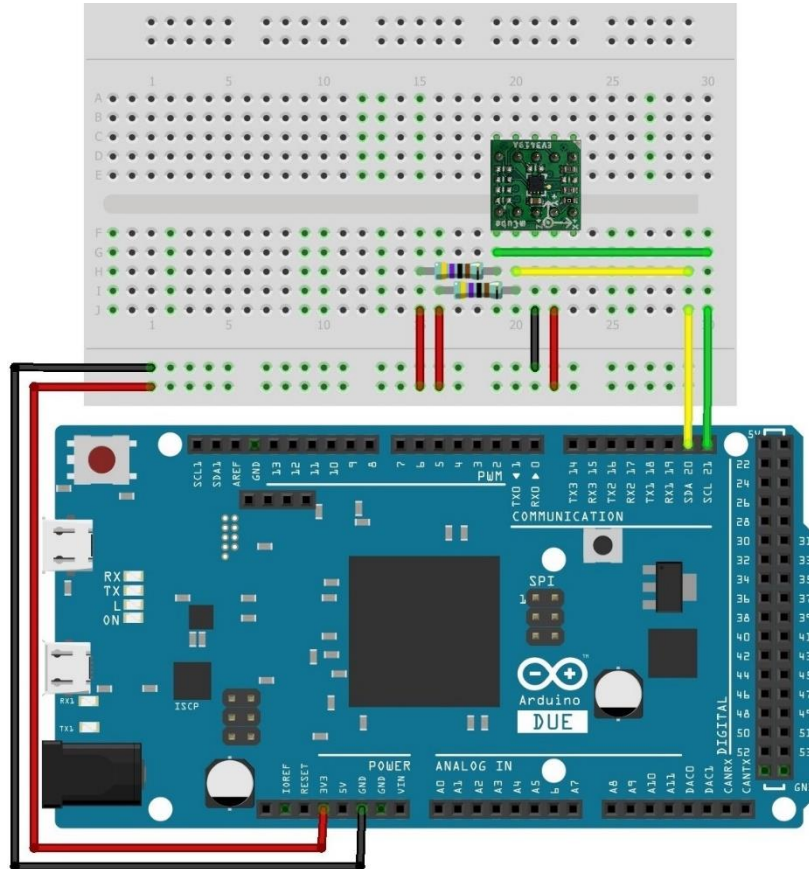
**INT** - HW interrupt signal pin. This pin will be triggered by the device when data is ready to read, or a motion event is detected by the accelerometer. (Not currently supported in the library for the interrupt pin, so please check the datasheet for the I2C commands and related registers).

**R6**: If using the sensor interrupt signal as open-drain, then install pull-up resistor  $\sim 4.7\text{K}\Omega$  into R6 (not installed by default).

## 2 ASSEMBLY AND TEST

Please note that the SPI and I2C interfaces cannot both be active at the same time as the clock (SCK) and data (SDA) are shared between the two protocols.

### 2.1 I2C INTERFACE

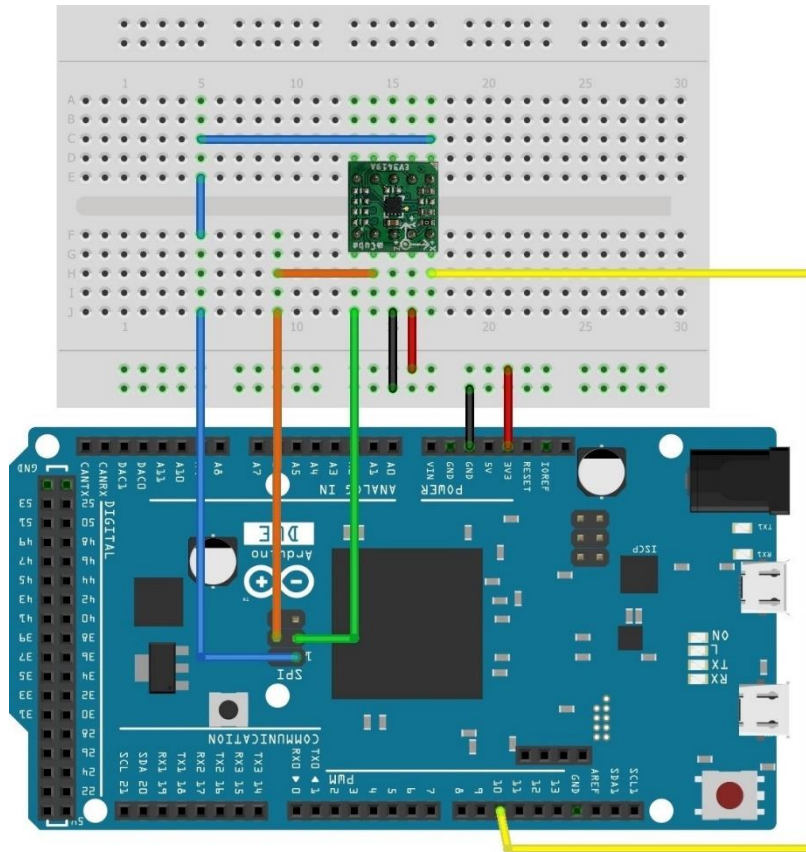


The EV3419A evaluation board can be easily wired to any microcontroller. This example shows a typical Arduino DUE platform. For other microcontrollers, be sure it has I2C with repeated-start support, then port the code. Please refer to the illustration below to connect the related pins.

- Connect **DVDD** to the power supply, **3.3V**. (WARNING: attempting to power the part at a voltage exceeds the maximum rating of 3.6V is likely to damage it.)
- Connect **GND** to common power/data ground.
- Connect the **SCL** pin to the I2C clock **SCL** pin on your Arduino.
- Connect the **SDA** pin to the I2C data **SDA** pin on your Arduino.

The **MC3419** has a default I2C address of **0x4C** and it can be changed to **0x6C** by tying the **SDO** pin to **VDD**.

## 2.2 SPI INTERFACE



The EV3419A evaluation board can be easily wired to any microcontroller. This example shows a typical Arduino DUE platform. Please refer to the illustration below for connecting the related pins and then port the code to get the raw X, Y, Z sensor data.

- Connect **DVDD** to the power supply, **3.3V**. (WARNING: attempting to power the part at a voltage exceeds the maximum rating of 3.6V is likely to damage it.)
- Connect **GND** to common power/data ground.
- Connect **SCL** to ICSP-3 as Serial Clock.
- Connect **SDO** to ICSP-1 as Master Input, Slave Output.
- Connect **SDA** to ICSP-4 as Master Output, Slave Input.
- Connect **SCS** to digital I/O pin **10** as Slave Chip Select.

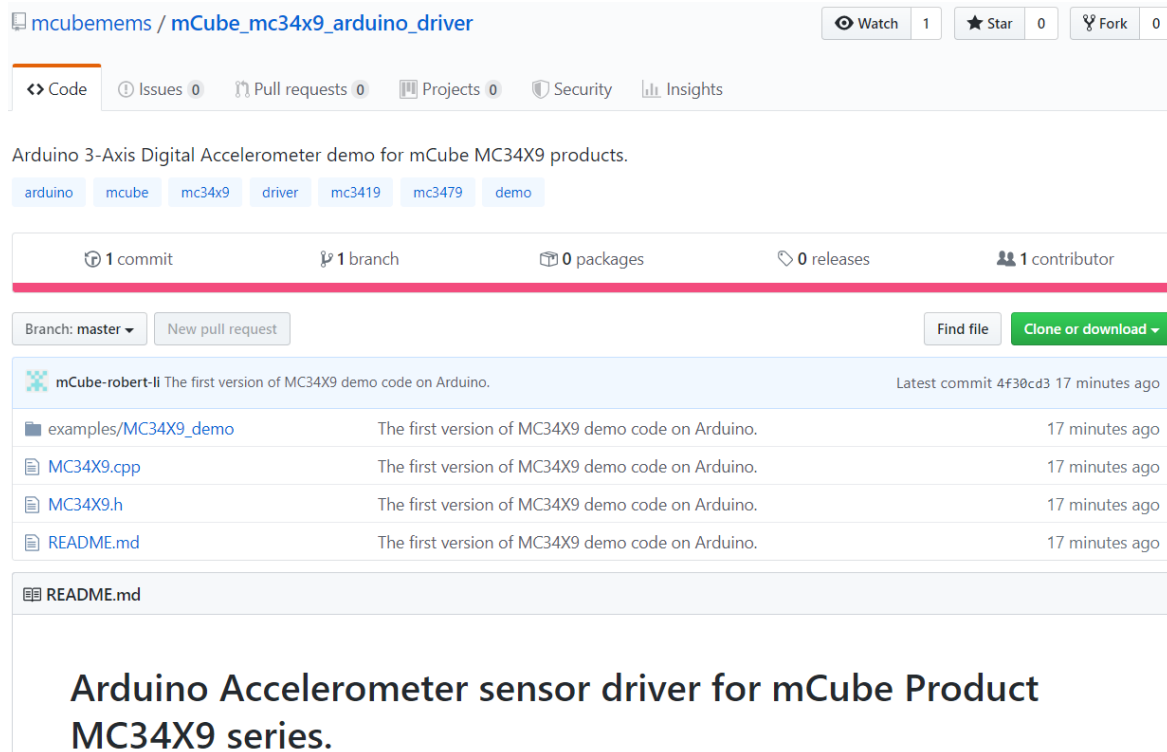


## 3 DEMO

### 3.1 DOWNLOAD THE DRIVER FROM GITHUB

To begin reading sensor data, you will need to download the MC3419 Library from the GitHub repository and simply click URL to clone or download the driver file.

[https://github.com/mcubemems/mCube\\_mc34x9\\_arduino\\_driver](https://github.com/mcubemems/mCube_mc34x9_arduino_driver)



mcubemems / mCube\_mc34x9\_arduino\_driver

Watch 1 Star 0 Fork 0

Code Issues 0 Pull requests 0 Projects 0 Security Insights

Arduino 3-Axis Digital Accelerometer demo for mCube MC34X9 products.

arduino mcube mc34x9 driver mc3419 mc3479 demo

1 commit 1 branch 0 packages 0 releases 1 contributor

Branch: master New pull request Find file Clone or download

mCube-robert-li The first version of MC34X9 demo code on Arduino. Latest commit 4f30cd3 17 minutes ago

|                      |                                                   |                |
|----------------------|---------------------------------------------------|----------------|
| examples/MC34X9_demo | The first version of MC34X9 demo code on Arduino. | 17 minutes ago |
| MC34X9.cpp           | The first version of MC34X9 demo code on Arduino. | 17 minutes ago |
| MC34X9.h             | The first version of MC34X9 demo code on Arduino. | 17 minutes ago |
| README.md            | The first version of MC34X9 demo code on Arduino. | 17 minutes ago |

README.md

**Arduino Accelerometer sensor driver for mCube Product MC34X9 series.**

Download with ZIP format from link above and check the zip file consisting of **MC34X9.cpp**, **MC34X9.h** and examples directory.

An excellent tutorial on Arduino library installation is located at:

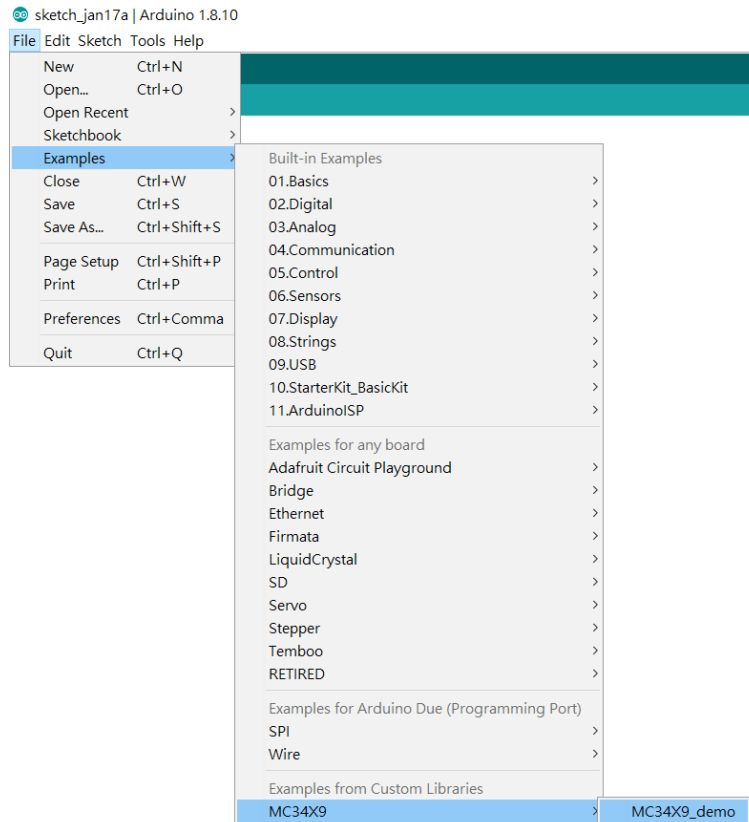
<https://www.arduino.cc/en/guide/libraries#toc4>

You may need to create the library subfolder if it is your first library files. Then just restart the IDE.



### 3.2 LOAD THE DEMO

Open File->Examples->MC34X9-> MC34X9\_demo and upload to your Arduino while it is wired to the sensor.



If you need the sensor running on SPI, please configure the bus as SPI in the MC34X9\_demo.ino shown as below. Otherwise, the default is I2C bus.

```
/** bSpi: I2C/SPI bus selection. SPI: 0, I2C: 1 */
const uint8_t bSpi = 1;
```

Setup I2C interface as 2.1 will get I2C address 0x4C.

Setup SPI interface as 2.2 will get SPI select pin 10.

```
/* Chip Select & Address */
uint8_t chipSelect = 0;
const uint8_t SPIChipSelectPin = 10; // SPI chipSelectPin
const uint8_t I2CAddress = 0x4c; // I2C address
```

Now open the serial terminal window at 115,200 baud rate speed to begin the test.

```

COM30
mCube Accelerometer MC34X9:
15:36:32.208 -> SPI mode
15:36:32.393 -> In GetRangeCtrl():
15:36:32.393 -> 20
15:36:32.393 -> Range: +/- 8 g
15:36:32.393 -> Low Power Mode SR
15:36:32.393 -> In GetCWakeSampleRate():
15:36:32.393 -> 17
15:36:32.393 -> Output Sampling Rate: 1000 Hz
15:36:32.393 ->
15:36:32.393 -> X:      174      Y:      21      Z:      4074      counts
15:36:32.393 -> X:      0.42      Y:      0.05      Z:      9.75      m/s^2
15:36:32.393 -> -----
15:36:32.393 -> Sensor FIFO enable.
15:36:32.393 -> Get interrupt: FIFO threshold.
15:36:32.393 -> X:      183      Y:      30      Z:      4107      counts
15:36:32.393 -> X:      0.44      Y:      0.07      Z:      9.83      m/s^2
15:36:32.393 -> -----
15:36:32.393 -> X:      156      Y:      51      Z:      4131      counts
  
```

☐ 自動捲動 ☒ Show timestamp NL(newline) 115200 baud Clear output

You will see the output from the serial terminal showing the current range scale and resolution of the sensor in the first three lines followed by two lines of output sensor data at some output data rate which depict “raw count” data for line 1: X: 174 Y: 21 Z: 4074 with 8G range, 16bit ADC resolution. Line 2 indicates the SI units for measuring acceleration as X: 0.42 m/s<sup>2</sup> Y: 0.05 m/s<sup>2</sup> Z: 9.75 m/s<sup>2</sup>.

To enable FIFO mode, which is controlled by the configurations below. FIFO size could be set up to maximum 31 samples. Enable FIFO threshold interrupt will automatically enable FIFO feature.

```

/**** FIFO control ****/
int FIFO_THRE_SIZE = 30;
/* FIFO Interrupt */
const bool enableFifoThrINT = false;
/* For FIFO feature, enable FIFO interrupt will automatically enable
FIFO feature */
bool enableFIFO = false;
  
```

To control embedded motion blocks, which is enabled by the configurations below. Enable individual motion mode will also enable corresponding interrupt.

***These modes can only be enabled separately.***

```
/** Motion control **/  
/* Enabling motion feature below also enables corresponded motion  
interrupt */  
const bool enableTILT = false;  
const bool enableFLIP = false;  
const bool enableANYM = true;  
const bool enableSHAKE = false;  
const bool enableTILT_35 = false;
```

## 4 LIBRARY REFERENCE

### 4.1 CREATE MCUBE\_MC34X9 OBJECT

You can create the MCUBE\_MC34X9 object with:

```
MC34X9 MC34X9_acc = MC34X9();
```

### 4.2 INITIALIZE AND CONFIGURE SENSOR

Initialize and configure the sensor with:

```
MC34X9_acc.start(bSpi, chipSelect);
```

Wake up sensor with your own configuration, it will follow the factory default setting:

```
MC34X9_acc.wake();
```

Stop sensor to change setting:

```
MC34X9_acc.stop();
```

### 4.3 SET RANGE

Set the accelerometer max range to  $\pm 2g$ ,  $\pm 4g$ ,  $\pm 8g$ ,  $\pm 16g$  or  $\pm 12g$  with:

```
MC34X9_acc.SetRangeCtrl(MC34X9_RANGE_8G);
```

### 4.4 READ RANGE

Read the current range with:

```
MC34X9_acc.GetRangeCtrl();
```

It returns: 0 for  $\pm 2g$ , | 1 for  $\pm 4g$ , | 2 for  $\pm 8g$  | 3 for  $\pm 16g$  | 4 for  $\pm 12g$ .

### 4.5 SET SAMPLING RATE

Set the accelerometer sampling rate with:

```
MC34X9_acc.SetSampleRate(MC34X9_SR_DEFAULT_1000Hz);
```

## 4.6 READ SAMPLING RATE

Read the current sampling rate with:

```
MC34X9_acc.GetSampleRate ();
```

It returns sampling rate from 25 ~ 1000 Hz.

## 4.7 CONFIG MOTION FEATURE

To configure motion feature, open MC34X9.h

Set the threshold and debounce value used by the tilt and flip logic for activity detection:

```
#define s_bCfgFTThr          100
#define s_bCfgFTDebounce     50
```

Set the threshold and debounce value used by the any motion logic for activity detection:

```
#define s_bCfgANYMThr        200
#define s_bCfgANYMDebounce   100
```

Set the threshold, duration and count value used by the shake logic for activity detection:

```
#define s_bCfgShakeThr        300
#define s_bCfgShakeP2PDuration 10
#define s_bCfgShakeCount      1
```

Set the threshold and duration value used by the tilt35 logic for activity detection:

```
#define s_bCfgTILT35Thr       20
#define s_bCfgTILT35Timer     MC34X9_TILT35_2p0
```

## 4.8 CONFIG INTERRUPT MODE

Configure the FIFO interrupt mode with:

```
MC34X9_acc.SetFIFOINTCtrl(false, false, enableFifoThrINT);
```

Configure the motion interrupt mode with:

```
MC34X9_acc.SetINTCtrl(enableTILT, enableFLIP, enableANYM, enableSHAKE,  
enableTILT_35);
```

MC34X9 has 5 motion interrupt modes – TILT | FILP | ANYM | SHAKE | TILT35.

***These modes can only be enabled separately.***

## 4.9 READ RAW COUNT DATA

Read the raw count data and SI unit measurement with:

```
MC34X9_acc.readRawAccel();
```

## 5 DOWNLOADS

### 5.1 MC3419 ACCELEROMETER DATASHEET AND QUICK START GUIDE

<https://mcubemems.com/product/mc3419-3-axis-accelerometer/>

### 5.2 MC34X9 DRIVER AT GITHUB

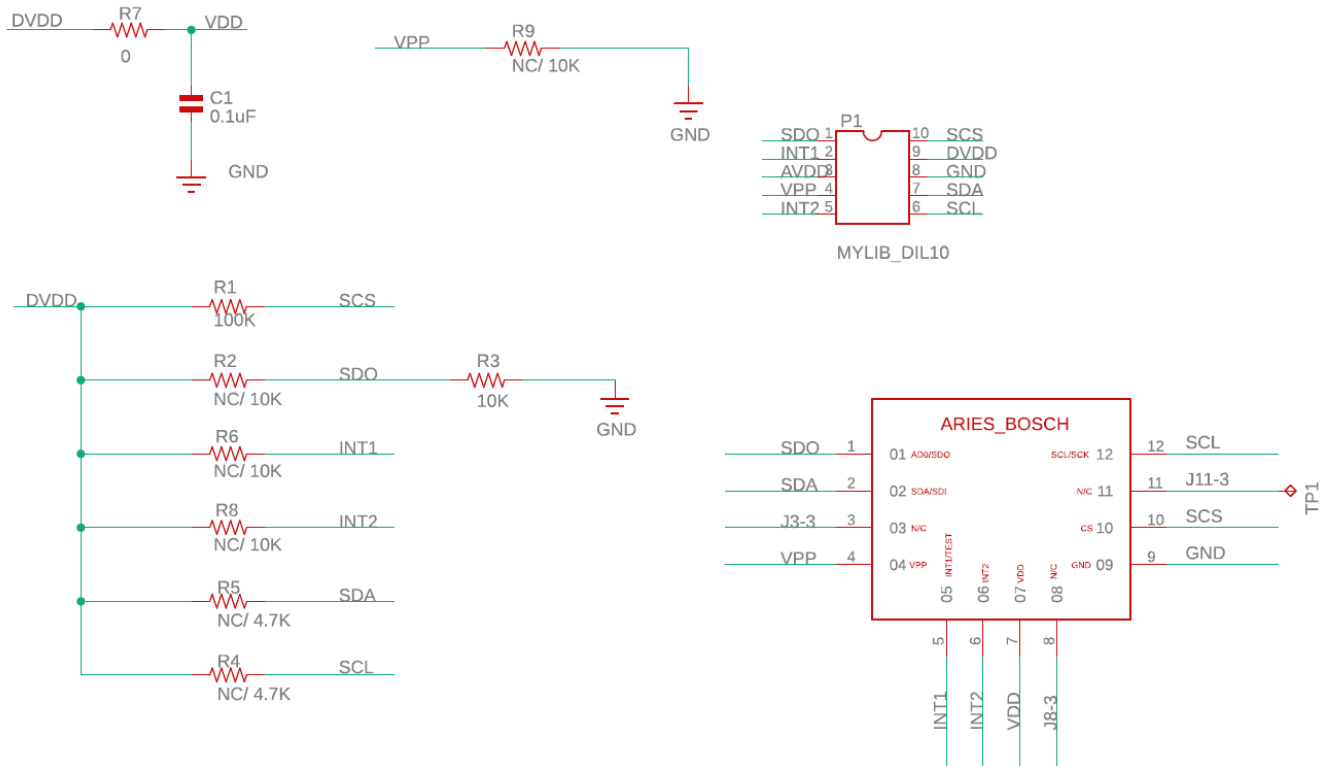
[https://github.com/mcubemems/mCube\\_mc34x9\\_arduino\\_driver](https://github.com/mcubemems/mCube_mc34x9_arduino_driver)

### 5.3 ALL OTHER MCUBE DOCUMENTATION

<https://mcubemems.com/resources-support/resources/>



## 6 SCHEMATICS



Above is a schematic on EV3419A. This is the factory preset when receiving the part.  
For other options, please refer to the following table:

| Interface                                  | R1    | R2               | R3   |
|--------------------------------------------|-------|------------------|------|
| SPI or I2C 0x4C ( <b>Factory default</b> ) | 100KΩ | DNI <sup>1</sup> | 10KΩ |
| SPI or I2C 0x6C                            | 100KΩ | 10KΩ             | DNI  |

**R4, R5:** Install ~4.7KΩ as the I2C pull-ups if using I2C mode and there is no I2C pull-ups installed. (DNI by default)

Note: It is recommended not to install more than one pull-ups per I2C bus.

**R6:** Install ~4.7KΩ pull-up resistor if setting the sensor interrupt pin to open-drain. (DNI by default)

**R7:** Sensor's driving current can be measured by putting an ammeter in place of R7.

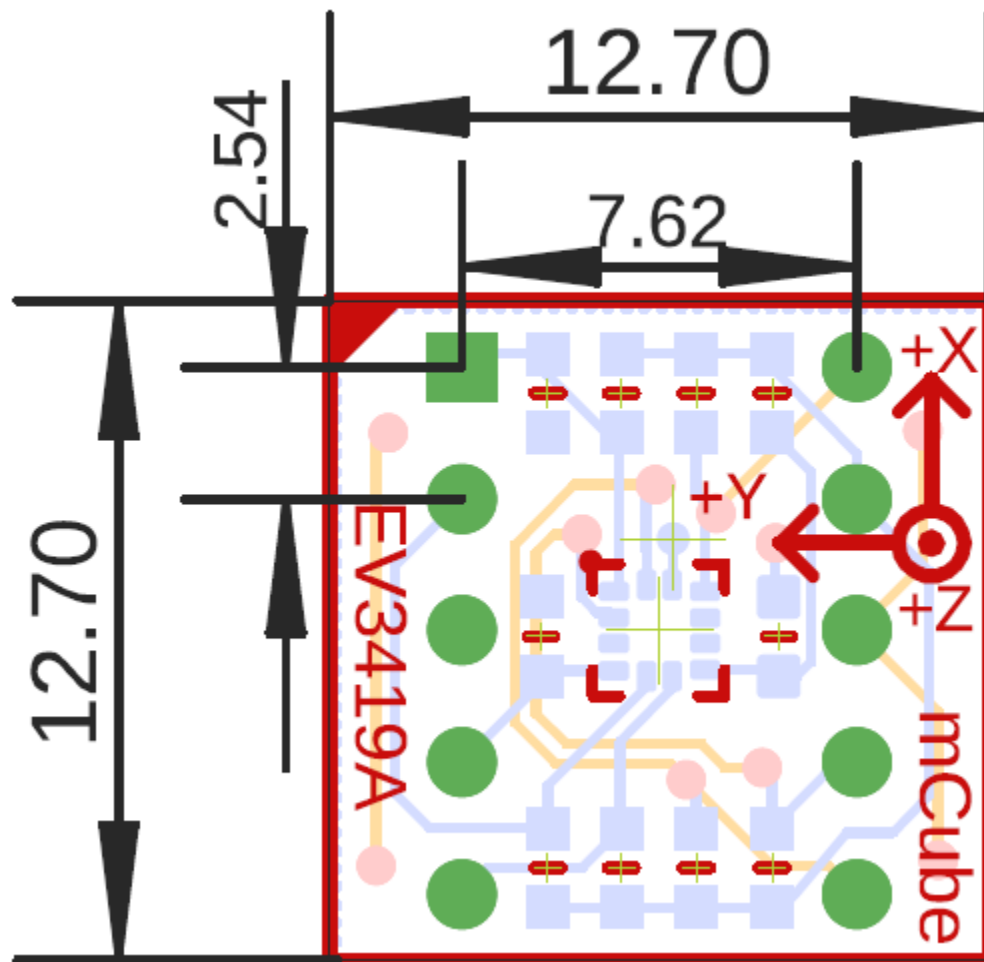
The physical location of the resistor is in the diagram in Section 8.

<sup>1</sup> DNI: Do Not Install

## 7 BILL OF MATERIALS

| Item | Part | Value   | Package          | Vendor | P/N          | Install |
|------|------|---------|------------------|--------|--------------|---------|
| 1    | C1   | 0.1uF   | CAP-0603         | Walsin | 0603B104K500 | Yes     |
| 2    | J1   | DIL10-A | DIL10-A          | -      | -            | Yes     |
| 3    | R1   | 100K    | RES-0603         | Walsin | WR06X103JTL  | Yes     |
| 4    | R3   | 10K     | RES-0603         | Walsin | WR06X102JTL  | Yes     |
| 5    | R7   | 0R      | RES-0603         | Walsin | WR06X000PTL  | Yes     |
| 6    | U1   | MC3419  | LGA-12_2.0 x 2.0 | mCube  | MC3419       | Yes     |
| 7    | R2   | 10K     | RES-0603         | Walsin | WR06X472JTL  | No      |
| 8    | R4   | 4.7K    | RES-0603         | Walsin | WR06X472JTL  | No      |
| 9    | R5   | 4.7K    | RES-0603         | Walsin | WR06X472JTL  | No      |
| 10   | R6   | 4.7K    | RES-0603         | Walsin | WR06X472JTL  | No      |
| 11   | R9   | 10K     | RES-0603         | Walsin | WR06X472JTL  | No      |

## 8 FABRICATION PRINT



NOTE: All dimensions are in millimeters.

## 9 REVISION HISTORY

| Date       | Revision         | Description    |
|------------|------------------|----------------|
| 2020-02-05 | APS-045-0033v1.0 | First release. |
|            |                  |                |
|            |                  |                |
|            |                  |                |
|            |                  |                |

## 10 LEGAL

1. M-CUBE reserves the right to make corrections, modifications, enhancements, improvements and other changes to its products and to this document at any time and discontinue any product without notice. The information contained in this document has been carefully checked and is believed to be accurate. However, M-CUBE shall assume no responsibilities for inaccuracies and make no commitment to update or to keep current the information contained in this document.
2. M-CUBE products are designed only for commercial and normal industrial applications and are not suitable for other purposes, such as: medical life support equipment; nuclear facilities; critical care equipment; military / aerospace; automotive; security or any other applications, the failure of which could lead to death, personal injury or environmental or property damage. Use of the products in unsuitable applications are at the customer's own risk and expense.
3. M-CUBE shall assume no liability for incidental, consequential or special damages or injury that may result from misapplication or improper use of operation of the product.
4. No license, express or implied, by estoppel or otherwise, to any intellectual property rights of M-CUBE or any third party is granted under this document.
5. M-CUBE makes no warranty or representation of non-infringement of intellectual property rights of any third party with respect to the products. M-CUBE specifically excludes any liability to the customers or any third party regarding infringement of any intellectual property rights, including the patent, copyright, trademark or trade secret rights of any third party, relating to any combination, machine, or process in which the M-CUBE products are used.
6. Examples of use described herein are provided solely to guide use of M-CUBE products and merely indicate targeted characteristics, performance and applications of products. M-CUBE shall assume no responsibility for any intellectual property claims or other problems that may result from applications based on the examples described herein
7. Information described in this document including parameters, application circuits and its constants and calculation formulas, programs and control procedures are provided for the purpose of explaining typical operation and usage. "Typical" parameters that may be provided in M-CUBE data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters including "Typical," must be validated for each customer application by customer's technical experts. In no event shall the information described be regarded as a guarantee of conditions or characteristics of the products. Therefore, the customer should evaluate the design sufficiently as whole system under the consideration of various external or environmental conditions and determine their application at the customer's own risk. M-CUBE shall assume no responsibility or liability for claims, damages, costs and expenses caused by the customer or any third party, owing to the use of the above information.



is a trademark of M-CUBE, Inc.

M-CUBE and the M-CUBE logo are trademarks of M-CUBE, Inc.,

All other product or service names are the property of their respective owners.

© M-CUBE, Inc. 2020. All rights reserved.